ROUTEGUARD

Software Requirements Specification

C11 Graph-Based Route Discovery & Network Connectivity Simulator

Document ID: RG-SRS-002 | Version: 0.2 Syllabus-aligned draft
Academic boundary: CSC2101 through Lecture 37; CSC2130 DSA Lab
Implementation language: ISO C11
Data scope: small, deterministic synthetic graph

IMPORTANT STATUS NOTE

This is a proposed specification, not proof of implementation. The route algorithm is BFS on an unweighted graph; post-Lecture-37 algorithms are excluded.

Document ID: RG-SRS-002

Version: 0.2 — Syllabus-aligned draft

Project status: Proposed; implementation not yet verified

Implementation language: ISO C11

Presentation layer: Minimal React/TypeScript web interface, optional and separate from the C algorithm engine

Academic constraint: DSA concepts scheduled through Lecture 37 of CSC2101; no later lecture topics are assumed in scope

Status and honesty note: This document defines the intended system. It is not evidence that the code, UI, or tests already exist. At report time, mark each requirement using the actual source code and test evidence. Do not claim unimplemented features as delivered.

Document control

Field	Value
Project title	RouteGuard: Graph-Based Route Discovery and Network Connectivity Simulator
Theory course	CSC2101 — Data Structures and Algorithms
Lab course	CSC2130 — Data Structures and Algorithms Lab
Primary user	Student/presenter and faculty evaluator
Network data	Synthetic, fixed, deterministic campus-style network
Algorithm engine	C11; no C++ source or C++ containers
Core topics	Graph representations, BFS, DFS, reachability, BFS shortest path on an unweighted graph, and complexity analysis
Approval state	Draft for cross-check against source code and course constraints

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the problem, scope, functional requirements, data model, algorithm contracts, interface behavior, error handling, tests, and acceptance criteria for RouteGuard. It is intended to guide implementation and let the project owner cross-check the final report and presentation against the actual program.

1.2 Project overview

RouteGuard models a small synthetic campus or facility network as an **undirected, unweighted graph**. Locations are vertices and walkways are edges. The user selects a source and destination, runs Breadth-First Search (BFS) to find a route containing the minimum number of edges, runs Depth-First Search (DFS) to explore the network or determine reachability, and disables a walkway to simulate a closure. RouteGuard reruns graph traversal and shows whether the destination is still reachable and how the set of reachable locations has changed.

The project is more than a static path display because users can compare adjacency-matrix and adjacency-list representations, inspect traversal order and operation counts, disable connections, and observe the resulting change in reachability. The system is educational; it is not intended for real emergency dispatch or safety-critical navigation.

1.3 Syllabus basis and Lecture 37 boundary

This scope is based on the supplied CSC2101 theory handout and CSC2130 lab handout:

- CSC2101 lecture-plan classes 23–28 cover graph introduction/applications, adjacency-matrix and adjacency-list representation, BFS for shortest path, DFS for path existence, and comparison of BFS/DFS.
- CSC2101 classes 33–37 cover performance analysis, time and space complexity, asymptotic notation, and performance analysis of iterative and recursive algorithms.
- CSC2130 lab handout identifies C as the lab programming language and lists graph representation in Class 9 and BFS/DFS in Class 10.

The scope boundary is the end of CSC2101 Lecture 37. The theory handout places the introduction to Divide-and-Conquer and Greedy approaches at Class 42. Therefore, RouteGuard does **not** depend on greedy shortest-path algorithms such as Dijkstra, priority queues/heaps for path finding, low-link bridge detection, articulation-point detection, or any other unverified post-boundary algorithm. Although some of those topics appear elsewhere in the complete course syllabus, this project does not rely on their being covered by Lecture 37.

1.4 Objectives

1. Model a practical route/network problem using graph vertices and edges.
2. Implement adjacency-matrix and adjacency-list representations in C11.
3. Implement BFS and DFS in C11 and present traversal results deterministically.
4. Use BFS to find a minimum-hop path in an unweighted graph.
5. Simulate a walkway closure and recompute reachability and the route.
6. Compare representation trade-offs and measured operation counts with theoretical complexity.
7. Provide a small, clear interface suitable for a live faculty demonstration.

1.5 Scope of the minimum viable product

- A built-in synthetic, undirected, unweighted network with stable vertex IDs and readable labels.
- C11 graph model and validation rules.
- Both adjacency matrix and adjacency list representations, derived from the same canonical fixture.
- BFS traversal and shortest-hop path reconstruction.
- DFS traversal and reachability/path-existence check.
- Reachable-vertex count and connected-component count using repeated traversal.
- Enable/disable an edge, recompute results, and restore the baseline graph.
- Operation counters for vertex visits, neighbor/edge checks, and queue/stack operations where applicable.
- A minimal web presentation layer if implemented; the C implementation remains authoritative for all graph algorithms.
- Native automated tests and a documented build/run workflow.

1.6 Explicit non-goals

- Weighted shortest paths, Dijkstra, A*, minimum spanning trees, or other greedy algorithms.
- Binary heaps, priority queues, bridge detection, articulation-point detection, low-link values, or advanced graph algorithms.
- Directed graphs, multi-graphs, dynamic route optimization, or concurrent mutations.
- Real maps, GPS, live traffic, external mapping APIs, real campus telemetry, or emergency-dispatch use.
- Accounts, authentication, database, cloud persistence, analytics services, or network uploads.
- Large-scale road-network performance claims; the initial graph is deliberately small and deterministic.
- Any DSA topic whose placement before Lecture 37 has not been confirmed by the supplied lecture plan.

1.7 Terms

Term	Meaning
Vertex	A graph node representing a location; identified by a unique integer ID
Edge	An undirected connection between two vertices
Active edge	A connection currently available for traversal
Disabled edge	A connection temporarily excluded from traversal to simulate closure
Path	A sequence of vertices connected by active edges
Minimum-hop path	A path with the fewest edges; BFS finds one in an unweighted graph
Reachability	Whether one vertex can be reached from another through active edges
Connected component	A maximal group of mutually reachable vertices in an undirected graph
BFS	Breadth-First Search; explores in layers using a queue
DFS	Depth-First Search; explores depth-wise using recursion or an explicit stack
V / E	Number of vertices / active edges (or all defined edges, where stated)
DSA	Data Structures and Algorithms
SRS	Software Requirements Specification

2. Overall description

2.1 Product perspective

RouteGuard has two separable layers:

1. **C11 algorithm engine:** owns the graph representation and performs validation, BFS, DFS, path reconstruction, component counting, edge-state changes, and operation counting.
2. **Presentation layer:** a small React interface (if implemented) renders the network and results, accepts source/destination choices, and requests operations from the C engine. A terminal interface may be used for initial implementation and testing.

If a WebAssembly bridge is adopted, it is a transport layer only. TypeScript must not silently reimplement BFS, DFS, graph connectivity, or path reconstruction. If WebAssembly is not built or tested, report the native C engine and frontend separately rather than claiming that the browser uses the C engine.

2.2 Users

User	Need
Student/presenter	Demonstrate graph modelling, BFS/DFS, edge closure, and complexity
Faculty/evaluator	Inspect the problem and question correctness, representation trade-offs, traversal behavior, and complexity
Developer/tester	Rebuild the C program, run deterministic tests, and verify interface results

2.3 Intended operating environment

- Native engine: compiler supporting ISO C11 and a supported build tool or documented compiler command.
- Optional frontend: Node.js/npm and locked frontend dependencies.
- Optional browser integration: a modern browser; Emscripten is required only to rebuild a WebAssembly artifact, not to run prebuilt assets if the clone-and-run setup is implemented and verified.
- The implementation must not require C++ compilation or C++ standard-library containers.

2.4 Design constraints

- Core source files use C11 (`-std=c11 -Wall -Wextra -pedantic` where GCC/Clang are available).
- Represent vertices/edges and dynamic structures using C structs, arrays, and clearly documented allocation/ownership rules.
- No third-party graph algorithm library performs BFS, DFS, path reconstruction, or component analysis.
- Use stable vertex IDs and deterministic neighbor iteration order so outputs and tests are reproducible.
- Traversal uses active edges only. A disabled edge remains recorded in the baseline data but is skipped by every graph operation.
- The route cost is the **number of edges**, not distance or travel time. The initial graph is unweighted.
- Timing is optional and informational; correctness tests should assert deterministic operation counts and results rather than exact elapsed time.
- Only synthetic example data is used.

3. Functional requirements

Statuses for tracking must be one of: **Not started**, **In progress**, **Implemented**, **Tested**, or **Deferred**. The default status for all requirements is **Not started** until verified.

3.1 Graph creation and validation

FR-001 — Load baseline fixture. The application shall load a deterministic synthetic network at startup.

FR-002 — Stable vertex IDs. Every vertex shall have a unique integer ID and a human-readable label. Labels shall not be used as identity keys.

FR-003 — Undirected edge model. Each edge shall connect two valid vertex IDs and be represented symmetrically in both supported graph representations.

FR-004 — Unweighted contract. Each active edge represents one hop. The system shall not display or imply a weighted travel time or shortest-time cost.

FR-005 — Duplicate/self-loop policy. The baseline fixture and any accepted edits shall reject duplicate undirected edges and self-loops, unless a documented change to this contract is approved.

FR-006 — Representation consistency. The adjacency matrix and adjacency list shall be built from the same canonical vertex/edge fixture and represent the same active graph state.

FR-007 — Validate vertices. A source, destination, or edge endpoint that does not refer to a valid vertex shall be rejected with a readable error.

FR-008 — Reset graph. The user shall be able to restore the baseline graph and clear simulated edge closures.

3.2 BFS, shortest-hop path, and DFS

FR-009 — BFS traversal. The system shall run BFS from a valid source using active edges only and return the traversal order.

FR-010 — Minimum-hop path. The system shall use BFS predecessor information to reconstruct a path with the minimum number of edges from source to destination in the unweighted graph.

FR-011 — DFS traversal. The system shall run DFS from a valid source using active edges only and return traversal order. DFS may use recursion or an explicit stack, with the chosen implementation and memory implications documented.

FR-012 — Reachability. The system shall report whether the destination is reachable from the source using active edges. DFS may be used for path-existence testing; BFS results may also establish reachability.

FR-013 — Unreachable destination. If no active path exists, the system shall report “No route found” (or equivalent) and shall not fabricate a route.

FR-014 — Source equals destination. If source and destination are the same valid vertex, the route shall contain that vertex and have zero edges.

FR-015 — Deterministic order. For the same graph, representation, and start vertex, traversal order shall be deterministic. Use a documented ascending vertex-ID order or another fixed order consistently.

FR-016 — Component count. The system shall compute the number of connected components by repeatedly starting a traversal from each not-yet-visited vertex.

FR-017 — Visited/reached output. The interface shall distinguish vertices reached from the selected source from vertices that remain unreachable.

3.3 Failure simulation and recovery

FR-018 — Disable edge. The user shall be able to mark a valid edge unavailable.

FR-019 — Re-enable edge. The user shall be able to restore a disabled edge.

FR-020 — Recompute after mutation. After an edge state changes, BFS, DFS, route, reachability, and component results shall use the updated graph state.

FR-021 — Impact summary. The system shall show whether the selected destination remains reachable and how the reachable-vertex count or component count changed.

FR-022 — Recover baseline. The user shall be able to reset all edge states without restarting the program.

FR-023 — No stale result. If the graph changes, the interface shall either recompute dependent results or visibly mark previous results as stale until rerun.

3.4 Comparing representations and showing metrics

FR-024 — Representation selection. The user shall be able to select adjacency-matrix or adjacency-list traversal for supported operations.

FR-025 — Representation consistency. Both representations shall return the same reachability result, component count, and minimum-hop count; when neighbor order is configured identically, traversal/path output should also agree. If different deterministic iteration policies are selected, the documentation shall explain that order can differ while correctness remains the same.

FR-026 — Operation counters. The C engine shall provide counters such as vertices discovered/processed, neighbor checks, queue insertions/removals for BFS, and stack pushes/pops (or recursive calls) for DFS. Definitions shall state exactly what is counted.

FR-027 — Complexity explanation. The interface or documentation shall explain the expected time/space trade-off: adjacency matrix uses O(V^2) storage and scans V possible neighbors per visited vertex; adjacency list uses O(V+E) storage and examines stored neighbors, giving O(V+E) traversal on a standard adjacency-list implementation.

FR-028 — Metrics source. Any displayed operation counters shall originate from the C implementation. Elapsed time, if displayed, shall be labelled informational and shall not be treated as deterministic proof of efficiency.

3.5 Interface and transparency

FR-029 — Graph view. If a web interface is delivered, it shall display labelled vertices and connections with distinct visual states for active and disabled edges.

FR-030 — Source/destination controls. The interface shall allow selection of valid source and destination vertices.

FR-031 — Algorithm controls. The interface shall provide actions for BFS, DFS, minimum-hop route, connectivity/component analysis, and graph reset.

FR-032 — Traversal/route display. Results shall show traversal order or the ordered sequence of vertices on the selected route.

FR-033 — Accessible state cues. Active/disabled edges and reached/unreached vertices shall not be distinguished by colour alone; labels, patterns, icons, or text must also convey state.

FR-034 — Errors. Invalid selections, missing vertices, and failed operations shall produce readable messages without crashing the program.

FR-035 — Synthetic-data disclosure. The app and documentation shall identify the default network as synthetic and educational.

3.6 Testing and documentation

FR-036 — Deterministic fixture. The repository shall contain a small graph fixture whose traversal orders, reachability, and shortest-hop routes can be checked by hand.

FR-037 — Unit tests. Graph creation, both representations, BFS, DFS, route reconstruction, component counting, reset, and edge disable/enable shall have automated tests.

FR-038 — Cross-representation tests. Equivalent matrix/list graph states shall agree on reachability, shortest-hop distance, and component count for all deterministic fixtures.

FR-039 — Regression tests. A single documented command or script shall run the core test suite from a clean build.

FR-040 — Accurate README. The README shall state the C11 requirement, commands actually tested, supported features, and the optional nature/status of the web/WASM interface.

4. Graph data model and invariants

4.1 Canonical model

The baseline graph is an undirected, unweighted graph G = (V, E). Vertices represent locations; edges represent available walkways. Each vertex has a stable integer ID and display label. Each edge joins two different vertex IDs and has an `active` flag. Weight fields are not part of the core model because every edge counts as one hop.

The canonical synthetic fixture is the source of truth. The adjacency matrix and adjacency list are derived from the same vertices and edges; changing an edge's `active` state must update both representations or update a single canonical state consulted consistently by both implementations.

4.2 Representation choices

Adjacency matrix: `matrix[u][v]` indicates whether an active edge exists between vertices `u` and `v` (or records edge existence separately from active state). For an undirected graph, matrix state is symmetric. Storage is O(V^2). Scanning all potential neighbors of one vertex takes O(V).

Adjacency list: each vertex has a list/array of its adjacent vertices. For an undirected graph, each edge is represented from both endpoints in traversal structures. Storage is O(V+E). BFS/DFS examine the actual stored neighbors.

The implementation must define how disabled edges are represented. A simple acceptable choice is to keep each canonical edge and its active flag, and have matrix/list traversal structures consult that state. An equally acceptable choice is to rebuild active matrix/list views after a mutation. The choice must be documented and tested for consistency.

4.3 Invariants

1. Every vertex ID is unique and within the graph's valid range.
2. Every edge endpoint references an existing vertex.
3. No self-loop or duplicate undirected edge exists under the MVP contract.
4. The graph is undirected: adjacency `u -> v` implies `v -> u` in each representation.
5. Matrix and list views represent the same edge and active-state set.
6. BFS marks a vertex visited before it can be enqueued a second time; each vertex enters the queue at most once per run.
7. DFS marks a vertex visited according to the documented traversal strategy and never recursively/iteratively explores a vertex indefinitely.
8. Every reconstructed route begins at the selected source, ends at the destination, and uses active edges only.
9. A BFS route in an unweighted graph minimizes the number of edges among all source-destination paths.
10. Disabling/re-enabling an edge changes later results but does not mutate vertex identity or reorder the canonical vertex set.
11. Reset restores the original edge-active state and default selections.
12. All dynamic memory is released on shutdown and on handled failure paths.

5. Algorithm contracts and complexity

5.1 BFS traversal

Input: graph representation, valid source ID, active-edge state.

Output: visitation order, visited flags, counters, and optionally predecessor/level arrays.

Core data structure: FIFO queue, which may be implemented using a C array with front/rear indices or an equivalent bounded/dynamic queue.

BFS visits vertices in increasing edge-distance layers from the source. If neighbors are inspected in ascending ID order, traversal order is deterministic. On adjacency lists, time is O(V+E) and auxiliary memory is O(V) for the visited, queue, predecessor, and distance arrays. On a matrix, time is O(V^2) due to scanning all candidate neighbors for each processed vertex; graph storage is O(V^2).

5.2 Minimum-hop path reconstruction using BFS

Because the graph is unweighted, BFS is the selected shortest-path algorithm. When a previously unvisited neighbor is discovered, record `predecessor[neighbor] = current`. After the search, walk backward from the destination via predecessors until the source is reached, then reverse the collected sequence.

If the destination is not visited, return a distinct unreachable result rather than an empty-looking valid route. The reported path length is the number of edges, which is one less than the number of vertices in a non-empty simple path. For source equal to destination, the path contains one vertex and zero edges.

This feature is a minimum-hop route, not a minimum-travel-time route. Adding non-uniform edge costs or Dijkstra is outside this SRS's Lecture 37 boundary.

5.3 DFS traversal and reachability

Input: graph representation, valid source ID, active-edge state.

Output: deterministic traversal order and visited/reachability result.

Implementation: recursive DFS or iterative DFS with a stack; select one and document stack-depth/resource behavior.

With adjacency lists, DFS time is O(V+E) and auxiliary memory is O(V), excluding the graph itself. With adjacency matrices, scanning all candidate neighbors for each processed vertex gives O(V^2) time and O(V) traversal auxiliary memory. DFS determines reachability/path existence but does not guarantee a minimum-hop route.

5.4 Connected components using traversal

To count components, initialize every vertex as unvisited. Scan vertices in stable ID order; whenever an unvisited vertex is found, increment the component count and start BFS or DFS from it. Every vertex is assigned to one component. With adjacency lists, total time is O(V+E); with a matrix, O(V^2).

This is an application built from the specified graph traversal techniques. It is not presented as a separate advanced algorithm from the syllabus.

5.5 Edge closure simulation

When a user disables an edge, set that edge's active state to false. Run the requested operation again using only active edges. A closure may leave the source and destination connected via another path or may make the destination unreachable. The application must report the observed result for the current synthetic graph; it must not assume every closure disconnects the graph. Re-enable and reset operations restore the appropriate state.

5.6 Comparison rules

- Use the same canonical graph, source, destination, active-edge state, and neighbor-order policy when comparing representations.
- Assert matching reachability, component count, and shortest-hop distance between matrix and list runs.
- If deterministic neighbor iteration is identical, assert identical traversal and reconstructed path output. If not, assert semantic correctness rather than exact order equivalence.
- Count logical operations using documented definitions. Do not compare counters that measure different things without explaining the difference.
- Do not use elapsed milliseconds as the principal correctness metric; timings are environment-dependent.

6. Interface requirements and proposed layout

6.1 Minimal user journey

1. Open RouteGuard; the default synthetic graph loads.
2. Choose a source and destination.
3. Run BFS shortest-hop route or inspect BFS/DFS traversal order.
4. Inspect the highlighted path, path length in edges, reachability status, and operation counters.
5. Disable one connection and run the search again.
6. Observe whether an alternative path exists and how reachability/component count changes.
7. Re-enable the edge or reset the graph.
8. Switch between adjacency matrix and adjacency list to compare behavior and complexity.

6.2 Minimal interface regions

- Header/title with an explicit [SYNTHETIC NETWORK] label.
- Graph visualization with readable vertex labels, active/disabled edge styles, and path highlighting.

- Source/destination selectors.
- Actions: BFS route, BFS traversal, DFS traversal, connectivity analysis, disable/enable edge, reset.
- Results panel with path sequence/hop count, traversal order, reachability, component count, and operation counters.
- Representation switch and a short matrix-vs-list complexity explainer.

The UI is a presentation layer. It may display C-produced results but must not claim that a web interface or WASM bridge works until it has been built and verified.

6.3 C-to-browser boundary (optional)

If using WebAssembly, expose a small C-compatible API with opaque state or explicitly documented integer IDs. Return serializable results or arrays with a defined ownership/lifetime. Keep C allocations and frees explicit. React components must not manipulate raw C structures or pointers. Prebuilt runtime assets may allow the end-user to run the interface without Emscripten, but the clean-clone behavior must be tested before it is claimed.

7. Non-functional requirements

NFR-001 — Language compliance. The graph algorithm engine shall be written in ISO C11. No C++ source, C++ compiler setting, or C++ container shall be required.

NFR-002 — Correctness. BFS, DFS, route reconstruction, and component counting shall satisfy the algorithm contracts in this SRS.

NFR-003 — Determinism. A fixed synthetic graph and fixed operation sequence shall produce the same route/traversal outputs and operation counts.

NFR-004 — Explainability. Every included algorithm, data structure, invariant, and complexity claim shall be explainable in a DSA viva and grounded in the supplied course plan through Lecture 37.

NFR-005 — No hidden algorithm substitution. The UI shall call the C engine if integration is implemented. Native results and UI results should be cross-checked through tests where applicable.

NFR-006 — Usability. Controls and results shall be labelled clearly, with readable errors and clear active/disabled edge states.

NFR-007 — Accessibility. Keyboard-operable controls, visible focus, and non-colour-only status indicators shall be provided in the web interface.

NFR-008 — Portability. Native C code shall use standard C11 facilities and avoid platform-specific APIs unless documented.

NFR-009 — Maintainability. Graph representation, traversal algorithms, fixture data, interface, and tests shall be separated into understandable modules.

NFR-010 — Testability. Core algorithms shall be testable without starting the frontend or browser.

NFR-011 — Privacy. The project shall use only synthetic demo data and shall not transmit graph data to an external service.

NFR-012 — Honest reporting. README, SRS, project report, presentation, and repository description shall distinguish implemented, tested, planned, and deferred functionality.

8. Error handling and edge cases

The implementation and tests shall address:

1. Empty graph (if the public API permits one).
2. One vertex and no edges.
3. Source equal to destination.
4. Invalid source or destination ID.

5. Destination disconnected from source.
6. Graph with multiple connected components.
7. Cycles and alternate routes between a source and destination.
8. A disabled edge that is the only route to a destination.
9. A disabled edge when an alternate route remains available.
10. Disable an already-disabled edge and enable an already-active edge; define whether this is a harmless no-op or a user-visible validation message.
11. Reset after several edge-state changes.
12. Matrix/list edge-state consistency after repeated toggle operations.
13. Deterministic neighbor ordering when a vertex has multiple neighbors.
14. Queue/stack boundary and cleanup behavior.
15. Allocation failure where safely injectable, or explicitly record it as untested.
16. Invalid input and clean shutdown in the native interface.

9. Verification and acceptance criteria

AC-001 — Language: the algorithm engine compiles as C11 with documented warning flags; the build does not require a C++ compiler.

AC-002 — Representation: the matrix and list encode the same baseline graph and remain equivalent after edge toggles/reset.

AC-003 — BFS: traversal order matches expected output on the hand-verifiable fixture.

AC-004 — Minimum-hop route: BFS returns a valid source-to-destination path with the minimum number of edges; no route is returned for an unreachable destination.

AC-005 — DFS: traversal order and reachability results match expected outputs on deterministic fixtures.

AC-006 — Component analysis: reported component count and vertex grouping match known results before and after edge closures.

AC-007 — Failure simulation: disabling and restoring edges changes later results correctly and does not leave stale graph state.

AC-008 — Representation comparison: matrix and list implementations agree on semantic results and have complexity explanations consistent with the actual implementation.

AC-009 — Operation metrics: counters are sourced from C, deterministically tested where possible, and their definitions are documented.

AC-010 — UI (if implemented): source/destination controls, traversal actions, edge toggling, reset, and result/error states work with the C engine.

AC-011 — Regression: all native tests pass from a clean build; any WASM tests and production browser build are reported separately.

AC-012 — Reproducibility: README commands work from a clean checkout, and required runtime assets are tracked if a frontend is delivered.

10. Test plan outline

Test ID	Scenario	Expected result
RG-GR-01	Load canonical fixture	Correct V and E counts; matrix/list agree

Test ID	Scenario	Expected result
RG-GR-02	Reject invalid endpoint	Readable failure; graph unchanged
RG-BFS-01	BFS from known start	Traversal order matches fixture
RG-BFS-02	Known source/destination	Valid minimum-hop route and edge count
RG-BFS-03	Unreachable destination	Explicit no-route result
RG-BFS-04	Source equals destination	One-vertex path, zero edges
RG-DFS-01	DFS from known start	Expected deterministic traversal order
RG-DFS-02	Reachability in disconnected graph	Correct reachable/unreachable result
RG-COMP-01	Count components on baseline fixture	Exact expected component count
RG-FAIL-01	Disable only connecting edge	Destination becomes unreachable if no alternate path exists
RG-FAIL-02	Disable redundant edge	Destination remains reachable if an alternate path exists
RG-FAIL-03	Re-enable then reset	Baseline results restored
RG-REP-01	BFS via matrix and list	Same reachability and minimum-hop distance
RG-REP-02	Toggle edge then compare representations	Both reflect same active graph
RG-MET-01	Run fixed fixture twice	Same operation counters
RG-ERR-01	Invalid source/destination	Clear error; no crash
RG-MEM-01	Repeated graph init/reset/free	No invalid access; allocation failures tested or documented as untested

These tests define intended verification. They are not claimed as executed until test output exists.

11. Complexity reference for the report

Let V be the number of vertices and E the number of edges in the graph.

Operation	Adjacency matrix	Adjacency list
Storage	O(V^2)	O(V+E)
Scan neighbors of one vertex	O(V)	O(degree(v))
BFS traversal	O(V^2)	O(V+E)
DFS traversal	O(V^2)	O(V+E)

Operation	Adjacency matrix	Adjacency list
BFS minimum-hop path	O(V^2)	O(V+E)
Connected components using repeated traversal	O(V^2)	O(V+E)
Disable/enable one existing edge	O(1) in canonical edge-state storage; representation refresh cost depends on design	O(1) if using a shared active flag; otherwise update cost depends on design

These are theoretical expected bounds for the described representations. The final report must confirm they match the implemented adjacency scanning, storage, and edge-state mutation strategy.

12. Requirement traceability template

Use this table while implementing and preparing the report. Replace blanks with actual code locations and real evidence.

Requirement group	Code location	Test/evidence	Status
Graph validation / matrix / list			Not started
BFS traversal / shortest-hop route			Not started
DFS / reachability			Not started
Connected components			Not started
Edge failure and reset			Not started
Operation counters / complexity view			Not started
Frontend (if implemented)			Not started
Native tests			Not started
WASM integration (if retained)			Deferred
Clean-clone workflow			Not started

Do not mark Tested without recorded test evidence. If implementation changes the SRS, update the SRS and report together rather than hiding the discrepancy.

13. Open decisions before implementation

1. **Graph size:** select a small deterministic network (recommended 10–20 vertices for demonstration) and record its exact edge list in the fixture.
2. **Neighbor order:** define ascending vertex ID or another stable order for matrix/list scans and tests.

3. **DFS implementation:** choose recursion or an explicit stack; document maximum depth or stack capacity.
4. **Mutation design:** decide whether edge active-state is stored once in a canonical edge array or duplicated in matrix/list structures.
5. **Frontend and WASM:** verify toolchain availability before promising WebAssembly integration.
6. **Demonstration route:** pick a source/destination that has one initial minimum-hop route and a useful failure scenario with a clear alternate/no-path outcome.
7. **Syllabus check:** keep the DSA implementation limited to concepts in or before CSC2101 Lecture 37. If the scope is expanded later, record explicit approval first.

14. Source basis

1. **Manipal University Jaipur, CSC2101 — Data Structures and Algorithms, Course Handout (July–December 2026), Section G, Lecture Plan.** Classes 23–28 cover graph types, graph representations, graph applications, BFS shortest path, DFS path existence, and comparison of BFS/DFS. Classes 33–37 cover performance analysis and complexity topics. Class 38 begins recurrence-solving methods; Class 42 begins Divide-and-Conquer and Greedy approaches.
2. **Manipal University Jaipur, CSC2130 — Data Structures and Algorithms Lab, Course Handout (July–December 2026), Sections B and G.** Section B identifies C as the lab programming language. Class 9 covers adjacency-matrix/adjacency-list representations; Class 10 covers BFS and DFS graph traversal.

Use the supplied handouts as the controlling academic source. Where a feature is inferred from a taught concept (for example, connected components by repeated traversal), label it as an application of BFS/DFS rather than claiming it is a separately named lecture topic.